Document number: P2080R0
Audience: LEWG

Ville Voutilainen
2020-01-13

Polymorphic allocators: There is no such thing as One True Vocabulary Type

Abstract

There is no such thing as One True Vocabulary Type..

..because there are multiple vocabularies.

Polymorphic allocators, and types using them, have two aspects:

1. The static part, aka the propagation traits and scoping
2. The dynamic part, aka the mechanism, memory_resource

Types like `std::vector` have both of these aspects intact. Types like the envisioned allocator-aware `std::function` and `std::optional` do not. The latter is a mistake.

Instead of adding library types that work with just `std::polymorphic_allocator`, (and thus hard-code the propagation and scoping), we should split such facilities into a generic part that has an allocator as a template parameter, and a more-specific part (an alias, perhaps) that packages a common allocator (perhaps a `polymorphic_allocator`) with the generic part.

When standardizing utility types, we should consider standardizing nice packages for common scenarios, but we should not forget to standardize extensible and customizable building blocks.

Propagation

A `std::polymorphic_allocator` is fine for the vocabulary it intends to serve. However, that vocabulary, where POCMA is false, is not suitable for all use cases under the sun. That vocabulary has two problems:

1. Shuffling a pmr::vector of pmr::string will always perform copies and will not perform moves. This is because in order to meet exception-safety guarantees, `move_if_noexcept` will copy, because the allocators are not always equal and they don't propagate.
2. Other operations where the user can guarantee that the allocator's memory arena outlives object instances will not propagate the allocator either, because SOCCC returns a `polymorphic_allocator` wrapping a default memory resource.

This is problematic. I can write a Major Architectural Sub-System (MASS) where I have a MASS-specific memory arena shared by all objects living under the control of that MASS. All propagation within the MASS is always fine. Yet `polymorphic_allocator` will defeat this idea since its POCMA and SOCCC will result in a default arena being used at the slightest provocation.

It seems that the argument is that if I would transfer an object out of a function when the allocator of the object is using a local arena, the allocator should not try to propagate because the arena will have expired. Another argument seems to be that between two different MASSes, the arenas would be different and therefore allocators should not propagate.

That's all fine and good. That's what propagation traits are for. If I have a local object using a local arena, I will give it an allocator that does not propagate. If I expose allocator-aware objects between two different MASSes, I will give those MASSes different allocator types, so objects will not just propagate from

```
std::vector<Foo, Mass1Allocator> get_stuff();
```

to a

```
std::vector<Foo, Mass2Allocator> my_local = Mass1::get_stuff();
```

In other words: both the static and dynamic aspects of an allocator/mechanism combination serve a purpose. Please don't drop either of those aspects when designing allocator-aware non-container types, like `function` or `optional`.

More about the static and dynamic aspects

In the pseudo-example above, Mass1Allocator and Mass2Allocator are BOTH polymorphic. They are, however, not `std::polymorphic_allocator`, but rather reusing it as an implementation detail. The allocators propagate, because they propagate inside MASS 1 and inside MASS 2, but not across the MASS boundary.

I can still do all the good things that memory_resources allow me to do without "infecting" a type. I can have multiple different memory resources as the mechanism of a std::vector<Foo, Mass1Allocator>. What Mass1Allocator allows me to do is define propagation that is different from `std::polymorphic_allocator`, and what Mass1Allocator and Mass2Allocator allow me to do is to have two different polymorphic-allocator-aware types that are not implicitly interoperable.

In other words, the best of both worlds.

Right, but what does this have to do with function and optional?

Well, if `function` and `optional` support only `std::polymorphic_allocator`, I lose the aforementioned abilities. I need to write my own allocator-aware function and optional to re-gain the static aspects of allocators. That would seem unfortunate.

So, what should we do?

Instead of

```
template <class T>> class __alloc_optional {...};
template <class T> using optional = metaprogram_select<T, __alloc_optional<T>, optional<T>>;
```

Do this instead:

```
template <class T, class Alloc> class basic_optional {...};
template <class T> using optional = metaprogram_select<T, basic_optional<T, std::polymorphic_allocator<T>>, optional<T>>;
```

In other words, instead of

1. Have a hidden type for an allocator-aware type that uses `std::polymorphic_allocator`
2. Provide an alias that aliases either the hidden type or an allocator-unaware type

we do

1. Have a named standardized type for an allocator-aware type that uses any allocator
2. Provide an alias that aliases either that type with a `std::polymorphic_allocator`or an allocator-unaware type

Why all this complexity? Polymorphic allocators are nice and simple.

Polymorphic allocators are nice and simple, agreed. Even the package of `std::polymorphic_allocator` is nice and simple, for the vocabulary that works with it. For vocabularies that don't, we reach for other kinds of allocators. And we want those allocators to be customization points, and thus our allocator-aware types should use those customization points. We can still package a `pmr::optional` so that it out-of-the-box gives us an optional with a standard polymorphic allocator, but we shouldn't close the door of customizing the behavior of an allocator-aware optional.

Yes, this is more complex to specify. It's barely more complex to implement. It's not more complex to design. One of the main headaches of designing an allocator-aware optional was figuring out how it should propagate in various scenarios. A `basic_optional` is _simpler_ in that regard, to specify, and to design; its allocator propagates the way the propagation traits of the allocator tell it to propagate. Layering a `std::polymorphic_allocator` on top of such a design is easy, it just plops in without any ado. And the design retains the ability to change how the allocator behaves, by using the `basic_optional` 'directly', or rather as a building block.